

Codebase Reference — What Every File Does and Why

A file-by-file walkthrough of the repository, written so any file can be opened cold and immediately understood: what it does, why it exists in its own module rather than folded into a neighbor, and the specific design decision or bug behind anything non-obvious in it. Companion to `docs/09_interview_prep.md` (the “why” at the architecture level) — this is the “where, exactly, and how” at the file level.

Organized by directory, in the order the pipeline actually executes.

Top-level layout

<code>src/edp/</code>	the package – everything the pipeline actually runs
<code>scripts/</code>	one-off tools: dataset generation, training, conversion
<code>config/</code>	all tunable thresholds, in YAML, never inline in code
<code>data/</code>	KiCad sources, the built reference library, golden truth, the two evaluation drawings (<code>data/validation/</code>)
<code>tests/</code>	unit tests for the deterministic logic (not GPU models)
<code>docs/</code>	this file and its siblings
<code>outputs/</code>	gitignored – build artifacts (trained weights, the reference-embedding cache, eval runs)

`src/edp/pipeline.py` is the only file that knows the full stage order. Every other module takes typed objects in and returns typed objects out (`src/edp/types.py` defines all of them) with no knowledge of what ran before or after it — this is deliberate: any stage can be tested, replaced, or reasoned about in isolation.

`src/edp/` — root

`cli.py`

The `edp` command-line entry point (`python -m edp.cli` , or the `edp` console script from `pyproject.toml`). Five subcommands: `run` (process one image or a directory), `build-library` (pre-warm the DINOv2 reference embedding cache), `eval` (score predictions against a golden-truth directory), `serve` (launch the FastAPI demo). Each subcommand is a thin `_cmd_*` function that imports its real logic lazily (inside the function body, not at module top) specifically so `import edp.cli` itself stays fast and dependency-light — a user just running `edp serve` shouldn't pay the import cost of the YOLO/DINOv2 stack until the command that actually needs it runs. `_cmd_build_library` shares its caching code path with `ReferenceLibrary.build()` (`classify/library.py`) via an explicit `cache_path` parameter rather than hand-rolling its own `.npz` writer — an earlier version had two slightly different save formats that could silently invalidate each other.

config.py

Every tunable value in the pipeline, as typed Pydantic models (one per pipeline stage: `PreprocessConfig`, `LocalizeConfig`, `ClassifyConfig`, `TextConfig`, `WiresConfig`, `ValidateConfig`, `OutputConfig`, composed into one `Config`). `Config.load()` reads `config/default.yaml` and validates it against these models — a typo or wrong type in the YAML fails fast at startup, not silently mid-pipeline. The rule stated in the YAML's own header ("never inline in code") is enforced structurally: every numeric threshold anywhere in `src/edp` traces back to a field here, not a magic number buried in a function.

types.py

The shared vocabulary every stage module is allowed to import from — and the *only* file other stage modules import from each other through. Six dataclasses: `Terminal` (a connection point, with `direction_deg` for directional wire-snapping), `Candidate` (a pre-classification localized region, carrying YOLO's own class/confidence through if that's the localizer that produced it), `Symbol` (a classified component — includes `top_k` / `margin` / `evidence_trace` for explainability, additive fields that don't appear in the trimmed delivered JSON), `TextToken` (one OCR reading), `Net` (an N-way conductor — the primary connectivity object, see `wires/nets.py`), `ValidationResult`, and `DrawingResult` (the final assembled output of one pipeline run). No stage-specific logic lives here — only the shapes stages agree to pass between them.

pipeline.py

Stage orchestration — the one place that knows the full 8-stage order and *why* that specific order: OCR and junction-dot detection both run before localization (not just before their "natural" later stages) because a text glyph or a filled dot is locally just as geometrically "busy" as a real symbol and would otherwise become a false localization candidate; classification runs before the final text-*association* stage, but still needs a lightweight OCR *hint* per candidate for the text-prior evidence source, so it computes that hint itself (`nearby_token_text`) rather than waiting for the full association pass. Each stage is wrapped in a `_Timer` context manager that records wall- clock time into a `timing` dict returned alongside the result — this is what the demo UI's per-stage timing breakdown comes from, not a separate profiling pass.

eval.py

The `edp eval` harness. `evaluate()` matches one drawing's predicted symbols to its golden-truth symbols by IoU (≥ 0.5 , greedy best-IoU-first — not the Hungarian/optimal-assignment algorithm, a deliberately simpler choice not formally proven equivalent to it, but checked by hand against every matching this produced on D4/D5: at under ~20 candidates per side, with boxes that rarely overlap ambiguously between two different golden symbols, greedy and optimal assignment agree in every case actually inspected). Reports detection precision/recall/F1 and — critically, only on matched pairs — classification accuracy, plus every individual confusion (`predicted=X true=Y`), every false negative, and every false positive by id. `evaluate_set()` runs this across every drawing in a golden directory and `print_report()` formats the summary table plus the per-symbol breakdown that made every "here's exactly which symbol changed" claim in this project's history verifiable rather than asserted.

src/edp/preprocess/ — stage 1

binarize.py

`to_grayscale` (handles RGBA→BGR→gray) then `binarize`: adaptive (local-mean) thresholding, not a single global threshold, because scan/ render background can vary within one drawing — adaptive thresholding handles that without per-drawing tuning. `_remove_speckles` does denoise via connected-component area filtering, specifically *not* a morphological open — a 2×2 open kernel was tried first and erases every 1px-wide stroke outright (verified by inspecting the binarized output, where nearly all real line art vanished and only dots/text fragments survived), since these drawings' own strokes are exactly that thin. Component-area filtering only looks at how big a blob is, never how thin, so it removes literal noise specks without touching real strokes of any width.

deskew.py

`estimate_skew_deg` (via `cv2.minAreaRect` on foreground pixels, angle normalized to a small correction around 0 rather than snapping to the nearest 90°) and `deskew`, which applies the correction only if it's inside `deskew_max_angle_deg` (5° default) — small-angle correction only, by design, since these are digitally-generated drawings, not scanned pages where large rotations would be expected.

layers.py

`blue_layer_mask` — an HSV-range color mask isolating D5's blue-drawn conductors. Returns an all-zero mask on monochrome input (D4) rather than raising, and every caller is required to treat that as "no color signal available," never an error — this is the concrete implementation of the "opportunistic signal, never a hard dependency" rule: any stage that *required* color to be present would silently break on half the input.

src/edp/localize/ — stage 2 (symbol proposal)

proposals.py

The original, classical-CV localizer — skeleton branch/endpoint *density*, not the thick-vs-thin morphological separation the design started with (rejected after verifying empirically that these drawings draw symbols and wires at the *same* stroke width, so an erosion-based split either erases everything or barely separates anything). `_interest_points` skeletonizes and finds pixels with degree ≥ 3 (branch) or $= 1$ (endpoint) via a 3×3 neighbor-count convolution; `_density_map` box-filters that into a local density; connected components of the thresholded density become candidate regions. `_corner_count` is the direct fix for the dominant false-positive pattern found in the original audit: counting *distinct* corner blobs (not raw interest-pixel count) inside a candidate box, since a plain wire bend has exactly one corner and a real symbol clusters three or more — confirmed by auditing every false positive on D4 individually. `_tighten_to_ink` shrinks the loose density-blob box down to actual ink extent. `_strip_text` and

`_strip_dots` remove OCR token regions and junction-dot regions before density is computed, for the reason given in `pipeline.py` 's docstring. This module is not dead code even though YOLO is the shipped default — `pipeline.py` falls back to it automatically if no trained YOLO weights exist, preserving “always runnable end to end” from before any model is trained.

`morphology.py`

Two surviving functions from the original thick/thin design: `thick_symbol_mask` (erode-then-dilate with a kernel wider than typical wire thickness — thin strokes vanish, thick/enclosed shapes survive) and `thin_stroke_mask` (binary minus the thick mask = wire-only pixels), which `proposals.py` still uses for its `kind` hint (symbol vs. ambiguous, based on how much of a candidate region overlaps the wire mask). A third function that was planned alongside these (`close_dashed_boundaries` , for bridging D5's dashed SHIELD box before contour extraction) was removed as dead code — written, documented, but never actually wired into the localization path that shipped.

`merge.py`

`merge_overlapping` — unions any two candidate boxes whose overlap (relative to the smaller box) exceeds `candidate_merge_overlap` (0.5), repeating until no pair qualifies. Shared by *both* localizers, for two different reasons: the density-based one occasionally produces two adjacent boxes for one physical symbol (fragmented density blobs), and YOLO's own NMS doesn't catch every near-duplicate — two boxes from different anchor scales can each independently pass YOLO's NMS while being 1-2px apart on the same real symbol (observed concretely on D4's MOSFET and a transistor, each double-boxed, which cascaded into an over-merged connectivity net downstream). On merge, the *higher- confidence* duplicate's YOLO class/confidence is kept, not discarded — duplicates aren't always redundant noise, they can be independent votes at different confidences, and one real D4 case had a 0.27-confidence wrong vote and a 0.53-confidence correct vote for the same box.

`src/edp/detect/`

`yolo_detect.py`

The shipped localizer. `detect_candidates` loads the trained `.pt` weights (`@lru_cache` 'd so repeated calls in one process don't reload the model), runs inference at `yolo_conf_threshold` / `yolo_iou_threshold` , and — critically — extracts YOLO's own `box.cls` / `box.conf` per detection and carries them through on the `Candidate` object rather than discarding them (an earlier version treated YOLO as localization-only and threw this away; the docstring explicitly records that reasoning as *wrong*: YOLO is supervised and in-domain, trained on this project's own schematic symbols, and measurably right on cases DINOv2 alone gets wrong). Falls back to an empty candidate list, not an exception, if the weights file doesn't exist — the same “stay runnable with an incomplete model” pattern as the empty-reference-library case. Calls the shared `merge_overlapping` at the end for the near-duplicate reason described above.

src/edp/classify/ — stage 3, the largest and most architecturally significant subpackage

kicad_import.py

A complete, dependency-free `.kicad_sym` S-expression parser and rasterizer — no external Lisp/S-expression library, because the grammar KiCad actually uses (atoms, quoted strings, nested parens) is small enough for a ~30-line tokenizer plus recursive-descent parser to handle completely.

`parse_sexp` / `_tokenize` do the generic parsing; `_parse_graphic_subsymbol` walks the parsed tree extracting the specific primitives this project's fetched symbol files actually use (polyline, circle, rectangle, 3-point arc, pin) into a `KicadSymbol` dataclass. `render_symbol` rasterizes to a white-background PNG (KiCad's Y-up axis is flipped to image Y-down during projection) and — the single most consequential piece of logic in this file — computes each pin's true wire-contact point. **This is where the KiCad pin-geometry bug lived:** `(at x y angle)` is the pin's *outer* electrical connection point, and `length` walks *inward* toward the body from there; an early version had this backwards, so every rendered symbol's terminal template pointed at the component body instead of the true wire-attachment point, and fixing the two-point swap alone moved D4/D5 connectivity coverage from roughly half to over 70% with no other change. The code comment documenting this cites the exact verification (`R_US.kicad_sym` 's pin 1 coordinates) used to confirm the correct convention before fixing it. `_draw_arc` fits a circle algebraically through three image-space points to render KiCad's 3-point arc primitive, falling back to a straight polyline if the points are near-collinear (a circle fit would blow up numerically there).

embedder.py

Wraps DINOv2 (`facebook/dinov2-small` or `-base` , loaded lazily via `@lru_cache` so no model weights are fetched just from importing this module). `_pad_to_square` pads a crop to square with white before it reaches the HuggingFace processor — DINOv2's own `resize-then-center-crop` step would otherwise discard most of an elongated crop's long dimension (a resistor's tall/thin bbox, a capacitor's wide/short one) before the model ever sees it. `embed()` moves crops to GPU when available (a real, fixed bug: this used to run on CPU even with CUDA present, at ~50-70s per drawing; moving both the input batch and the model to `cuda` cut this to milliseconds per crop). Sets `KMP_DUPLICATE_LIB_OK=TRUE` defensively at import time — this Windows environment mixes conda-forge's MKL-linked NumPy/OpenCV with pip's official CUDA torch wheel, and both bundle their own OpenMP runtime; without this flag the second one to load trips a duplicate-runtime guard before torch finishes importing at all. The comment explains why this specific workaround is safe here (no true concurrent OpenMP use across the two runtimes in this process) rather than the "clean" fix (`conda install nomkl`), which was rejected as forcing a slow full-stack rebuild for no functional gain.

faiss_index.py

A thin wrapper around `faiss.IndexFlatIP` (exact inner-product search — cosine similarity, since embeddings are pre-normalized). The module docstring is explicit that FAISS buys nothing in raw speed at the current library size (a brute-force matrix multiply would be just as fast) — it's there so the *interface* is already correct for a size that would matter (many classes × rotation/mirror variants × multiple exemplars), so swapping to an approximate index later is a one-line change, not a rewrite.

`search(query, k)` returns empty arrays rather than raising on an empty index, matching the rest of the codebase's "degrade gracefully before the model/library is populated" pattern.

library.py

Builds the reference embedding library from `data/reference/<class>/*.png` — the single most performance-critical piece of infrastructure added late in the project. `ReferenceLibrary.build()` scans every class directory, augments each source image with the configured rotations (0/90/180/270) and an optional mirror (`_augment`, which also rotates the terminal template's normalized coordinates in lockstep — image and terminals must never drift out of sync), and embeds the result. `match()` returns the single nearest entry; `match_topk()` returns the top-*k* *distinct classes* (over-fetching raw neighbors first so many rotated copies of one class don't crowd out a genuinely different second candidate) — this feeds both the classification-evidence fusion and the per-symbol `top_k / margin` explainability fields. The **embedding cache** (`_cache_signature`, `_load_cached_embeddings`, `_save_cache`) is what turned a 45-85 second classify stage back into ~7 seconds once the library grew past ~250 entries: a SHA-256 signature over every source file's path/mtime/size plus the embedding model name and rotation config, stored alongside the embeddings in `<reference_dir>/index.npz`. Any real change — add, edit, or remove a reference image, switch models, change rotation settings — changes the signature and forces exactly one rebuild; nothing else does. Deliberately re-derives the meta comparison as a second, redundant check even though the signature alone should already guarantee correctness, specifically so "never silently serve embeddings for the wrong crop" is an explicit code-level guarantee, not an assumption.

evidence.py

The core abstraction the whole classification-fusion architecture is built on. `ClassificationEvidence` is one source's opinion: a `class_scores` dict (which need not cover every class or sum to 1), a `confidence` (how much *this specific reading* should be trusted), and free-form `metadata` for explainability. `no_evidence()` is the explicit "abstained" value — carrying a `reason` in its metadata so the fusion trace can distinguish "this source had nothing to say" from "this source said nothing confidently," which look identical as an empty dict but mean different things when debugging a wrong answer later. `fuse_evidence()` does the actual math: for every source that fired, `totals[class] += weight[source] × evidence.confidence × evidence.class_scores[class]`, then ranks by total. The returned `FusionResult` includes a `margin` (top-1 minus top-2, normalized to the winning score's own scale, so it means the same thing regardless of how many sources fired or how large their totals happen to be) — this margin is what `classify/match.py`'s ambiguity router reads to decide whether to invoke a geometry specialist.

text_prior.py

Turns OCR text near a candidate into `ClassificationEvidence`, config-driven from `config/reference_designators.yaml` — no knowledge encoded directly in this file's logic. `normalize_ocr_text` upper-cases, strips whitespace/punctuation, and applies a small, explicit table of *safe* character substitutions (currently just `$ → S`, since Tesseract reliably misreads the switch designator "S1" as "\$1" and a dollar sign has no legitimate meaning in a component label) — deliberately not general OCR error correction, which would risk turning a low-confidence reading into a confident-looking wrong one. `evidence_from_text` checks part-number patterns first (a substring search over the whole

normalized blob — specific enough that "T2BC547" correctly resolves via the embedded BC547 even though the designator prefix T alone is deliberately absent from the table), then checks each *individual* whitespace-separated token against the designator regex — not the whole joined blob at once. That token-independence is a fix for a real bug: an early version matched the joined string as one blob, and a single leading garbage token (a different Tesseract pass misreading the same region) could shift a perfectly good "R6" reading out of the regex's anchored-at-the-start match, silently defeating a correct answer. Unparseable text returns `no_evidence`, never a weak guess — a stated, load-bearing design rule.

`probe.py`

Loads a `scikit-learn` `LogisticRegression` artifact (from `scripts/train_linear_probe.py`) and exposes it as the `dinov2_probe` evidence source — reusing the *same* DINOv2 embedding `classify/match.py` already computed for the nearest-neighbor source, at zero extra cost. `_load` is `@lru_cache` 'd per model path; `probe_evidence` abstains cleanly (`no_evidence`) if no artifact exists at the configured path yet, the same graceful-degradation pattern used throughout.

`specialists.py`

The geometry-based confusion-pair resolvers — the only stage-3 evidence source that looks at raw pixels a second time after the initial classification pass. `CONFUSION_GROUPS` and `select_specialist` define which specialist handles which group; deliberately excludes the BJT_NPN/ BJT_PNP pair from routing (`npn_pnp_specialist` exists, is unit-tested, and is *not* reachable via `select_specialist` — a documented, measured decision, not an oversight: 2/2 correct on this project's own clean reference renders, 1/3 correct on real D4 crops). Each specialist function follows the same contract: take a crop, return `ClassificationEvidence` or abstain — never forced to produce an answer.

`battery_capacitor_specialist`: finds "plates" via `_find_plates`, which runs two independent passes — a per-row longest-contiguous-run scan for flat plates (immune to unrelated text sharing the same row, which an earlier width-fraction-based version was not), and a connected-component scan for curved plates (a swept arc's low fill-ratio, wide, non-tall bounding box distinguishes it from a filled shape). Decision: ≥ 4 plates $\rightarrow$ Battery; exactly 2, one curved $\rightarrow$ Capacitor_Polarized; 2 flat + a detected "+" mark (`_detect_plus_mark` — ink concentrated along both a component's middle row and middle column but not its corners) $\rightarrow$ Battery; 2 flat, no "+" $\rightarrow$ Capacitor.

`bjt_potentiometer_specialist`: `_detect_circle` via `cv2.HoughCircles` (param2=30, tuned up from a looser default that false-fired on this project's own Potentiometer reference render) $\rightarrow$ BJT family; `_has_rectangular_body` (contour fill-ratio + aspect ratio) as a secondary confirming check when no circle is found $\rightarrow$ Potentiometer.

`npn_pnp_specialist`: locates the emitter arrowhead as a small triangular contour (`cv2.approxPolyDP` tried across several epsilon values, since a *stroked*, not filled, arrowhead's outline zigzags enough that a single fixed epsilon sometimes returns 4-6 vertices instead of 3) and compares whether its farthest vertex from centroid sits farther from or closer to the circle center than the base — away = NPN, toward = PNP. Kept, tested, not routed, per the measured result above.

match.py

The orchestrator that turns one `Candidate` into one final `Symbol`. Builds four of the five evidence sources directly (`_yolo_evidence` , `_dinov2_evidence` , inline `probe_evidence` call, `evidence_from_text`), fuses them via `evidence.py` 's `fuse_evidence` , then calls `_select_ambiguous_specialist` to decide whether a fifth (geometry) vote is warranted — and if so, re-fuses with it included. That router deliberately does *not* implement the “top-1 vs top-2 margin below a threshold” rule the architecture doc originally proposed: measured directly against real cases, a literal margin check missed every one it was meant to catch (a confidently *wrong* answer has a large margin, not a small one; a spurious third candidate can edge out the true class for 2nd place, hiding it from a top-2-only check). The shipped rule fires whenever fusion's top-1 *or* ≥ 2 of the top-3 candidates fall in a known confusion group — a wider net that costs a little extra time on false-alarm cases but was the version that actually worked when checked.

`_instantiate_terminals` scales a matched reference entry's normalized terminal template onto the candidate's real-image bbox — this is the step that turns “this crop is a Resistor” into two real pixel coordinates a wire can later snap to.

src/edp/text/ — stage 4

ocr.py

`run_ocr` : upscales (Lanczos, `upscale_factor=3` — the single highest- impact OCR decision, since D5's labels are only a few pixels tall natively and Tesseract has an effective floor around $\sim 20\text{px}$ of glyph height), then runs *every* configured Tesseract PSM mode (`tesseract_configs` — three by default) across *every* configured orientation (0/90/270, since D4 sets a label vertically). Returns every reading from every pass, unmerged — an earlier version deduplicated overlapping readings by keeping the highest-confidence one, and that measurably lost a real answer (Tesseract's confidence for short garbled technical tokens isn't a trustworthy correctness signal), so passing every reading through and relying on downstream nearest-token matching to surface the correct one redundantly was kept instead.

`_ensure_tessdata_prefix` sets `TESSDATA_PREFIX` defensively at call time if unset — this environment's conda-forge Tesseract package never registers it itself, and without it Tesseract fails *closed* (returns zero tokens, no exception) rather than failing loud, which silently starves every downstream OCR-dependent stage without any visible error until specifically investigated.

associate.py

Post-classification instance/value resolution. `nearby_token_text` (the shared “closest tokens within `max_association_distance` , joined” logic) is exposed standalone specifically so `pipeline.py` can call it *before* classification too, for the text-prior evidence source — one function, two call sites, so the pre-classification hint and the final `ocr_text_raw` can never disagree about what counts as “nearby.”

`_assign_ids_greedily` is the fix for a real bug: assigning each symbol's id independently (nearest OCR token to *that* symbol) let two different nearby symbols both claim the same token (`"R4"` assigned twice), corrupting every downstream `connections` reference to that id. Fixed by collecting every (distance, symbol, id-candidate) triple across *all* symbols globally, sorting by distance, and greedily claiming each

token and each symbol at most once. `_assign_values` stays simple per-symbol matching, deliberately, since values (`"10K"` , `"50V"`) are not required to be unique the way instance ids are.

src/edp/wires/ — stage 5-6

skeleton.py

`subtract_symbols` zeroes out symbol bboxes (with a small pad) from the binary image;
`skeletonize_wires` runs `skimage.morphology.skeletonize` (morphological thinning to 1px width) on what's left. Deliberately minimal — the graph-building logic that used to live here as `skeleton_graph_nodes` (an endpoint/branch-point detector via neighbor- count convolution) was removed as dead code once `nets.py` 's own NetworkX-based traversal made it redundant; it was never actually called.

junctions.py

`detect_junction_dots` : finds filled dots by local ink *fill-ratio* within a small disk around every skeleton branch point, not by `cv2.findContours` shape matching. The docstring records exactly why the contour approach was rejected: a dot sitting on a wire (the normal case — a dot only exists where wires meet) is 8-connected to that wire, so the *whole dot-plus-wire run* becomes one contour, and `minEnclosingCircle` of that shape looks nothing like a small circle — this was silently dropping most real dots, catching only isolated ones (measured: ~30 on D5, far fewer than actually present). The fill-ratio approach is invariant to whatever else the point is connected to: a plain crossing only fills the width of its two crossing strokes, a drawn dot fills most of the disk regardless. Radius tuned to `junction_dot_max_radius - 1` specifically — the full max radius dilutes a small real dot's fill ratio below threshold, while the min radius false-fires on ordinary zigzag/coil corners (locally dense enough to fill a tiny disk too).

nets.py

The connectivity core, and the file implementing this project's single highest-leverage design decision: **the "crossing without a dot is not a connection" rule is enforced structurally, not as a post-hoc filter.**

`_pixel_graph` builds an 8-connected graph over skeleton pixels. `_find_crossing_regions` groups adjacent degree- ≥ 3 pixels into regions (a real crossing can span more than one skeleton pixel) and records each region's "arms" — the direction and outside-neighbor of every branch leaving it. `build_nets` then rewires the graph per region: if dotted (or fewer than 4 arms — a plain T-junction, treated as connected defensively, since an undotted T isn't a valid drawing convention here), collapse to one synthetic junction node joining all arms; if undotted with exactly 4 arms, remove the crossing pixels entirely and directly connect each arm to whichever other arm is closest to 180° opposite it (`_pair_by_opposite_angle`) — splitting one crossing into two independent straight-through paths *before* connected components are computed, so the two conductors land in genuinely different graph components with no correction step required afterward. `_attach_terminals` / `_snap_terminal` then snap each symbol's terminals to the resulting nets: when a terminal has a known pin direction (from the matched KiCad reference), the search is a narrow cone (`terminal_snap_cone_deg`) extending to

`terminal_directional_reach` (45px) rather than an isotropic circle — restricting the search direction is what allows a longer reach without the false-positive risk a correspondingly larger plain radius would carry in a dense drawing. Terminals with no known direction fall back to the isotropic `terminal_snap_radius` (30px).

src/edp/validate/

checks.py

Self-consistency validation — explicitly *not* an accuracy score, since (per the module docstring) no ground truth existed for the two drawings when this was written. Four structural checks: symbols with terminals but no net membership (`isolated_symbols`), symbols below `min_confidence` , terminals never snapped to any net (`unattached_terminals`), and nets with fewer than 2 terminals (`single_terminal_nets` — a net that connects nothing is almost always a tracing artifact). This still runs even now that a real golden-truth `edp eval` exists, as a fast, ground-truth-free sanity pass useful on any new drawing that doesn't have hand-verified labels yet.

src/edp/emit/ — stages 7-8

json_out.py

`to_json_dict` : exactly the four fields the brief's own example shows — `id` , `type` , `coordinates` , `connections` — via `_connections_for` , which expands a symbol's net memberships into the sorted list of other symbol ids sharing any of those nets. Nothing else. The richer internal representation (confidence, rotation, terminals, OCR provenance, per-net junction detail) is real and still drives both the graph construction and the validation checks — it's just never dumped into this specific output, since matching the brief's schema exactly was an explicit, later-stage simplification request.

graph_out.py

`build_bipartite_graph` builds the rich internal view directly from `DrawingResult` — symbol nodes and net nodes as distinct types, edges carrying terminal index — for the GraphML/node-link JSON exports. `build_component_graph_from_json` builds the *delivered* view straight from the trimmed JSON's `connections` field, not by re-deriving anything from nets or terminals — deliberately, so the delivered JSON and the delivered graph picture are visibly the same data, never two independent projections that could silently drift apart from each other. `_grid_of_components_layout` is a from-scratch NetworkX layout replacing a naive single `spring_layout` call, which let connected clusters visually collapse onto each other tightly enough to hide their own edges — verified the edges existed in the graph object but were invisible in the render. Laying out each connected component in its own local coordinate space, then tiling components on a grid, guarantees no cross-component overlap.

src/edp/web/

server.py

A thin FastAPI wrapper holding no pipeline logic of its own — `/` serves the static demo page, `POST /api/process` runs `pipeline.run()` on an uploaded file and returns the trimmed JSON plus three base64-encoded PNGs (original, a detection overlay drawn by `_render_detection_overlay`, and the graph) plus the per-stage `timing` dict straight from `pipeline.py`'s own `_Timer`. Uploaded files go to a `NamedTemporaryFile` and are cleaned up in a `finally` block regardless of success or failure.

static/index.html

The demo frontend: a 2×2 grid (original drawing / detected-symbols overlay / syntax-highlighted JSON / graph), fetching `/api/process` on file selection and rendering all four panels from one response. Pure HTML/CSS/JS, no build step or framework — appropriate for a page whose entire job is displaying one API call's result.

scripts/ — tooling, not part of the runtime pipeline

build_reference_from_kicad.py

Renders every `.kicad_sym` file in `data/kicad_raw/` into `data/reference/<class>/*.png` + `.terminals.json` sidecars, via `classify/kicad_import.py`. The `SOURCES` dict is the single place that maps a source filename to a target class name — adding a KiCad style variant to an existing class, or a source for a brand-new class, is a one-line addition here plus dropping the `.kicad_sym` file in `data/kicad_raw/`.

generate_synthetic_dataset.py

Composites reference symbols onto blank canvases to make YOLO training data, with auto-generated bounding-box labels (a byproduct of where the generator placed each symbol, never human annotation). `TARGET_SIZE_RANGE` (24-85px) was measured directly from D4/D5's real symbol sizes, not guessed. Adds Manhattan-routed wire clutter and small dash-cluster text clutter as unlabeled negatives, so the detector learns what's *not* a symbol without ever being told so explicitly. Parameterized (`--symbols-min/max` , `--canvas-min/max`) to support both a sparse "scatter" and a denser packing variant from one script.

generate_ladder_circuits.py

A second, topologically-realistic generator: closed rectangular border, top/bottom rails, vertical branches with 1-2 components in series, junction dots at rail intersections — built specifically to isolate "does the detector handle real circuit topology" from "does it generalize to unfamiliar symbol styles," which a single harder-scatter generator would have conflated into one uninterpretable score.

`domain_randomize.py`

Shared, semantics-preserving jitter (small rotation, stroke-width dilate/erode, blur, sensor noise, contrast/gamma, simulated JPEG re-encode) used by both the linear-probe training data generator and available for any future synthetic generator. Explicitly excludes non-uniform aspect distortion and rotation past a few degrees — either would change proportions some geometry specialists key off, or overlap with the *real* orientation handling the 0/90/180/270 augmentation already owns.

`generate_procedural_variants.py`

Parametric drawing (not KiCad-sourced) for structural parameters no single fixed symbol can express: battery cell count (2-6, correct alternating long/short plate convention), resistor zigzag peak count (3-6), and a curved-bottom-plate polarized capacitor matching this project's actual drawn convention rather than the KiCad reference's thickness-based one. Writes directly into `data/reference/<class>/` in the same PNG + `.terminals.json` format `build_reference_from_kicad.py` produces, so `ReferenceLibrary.build()` picks these up with no special handling.

`train_yolo.py`

Trains YOLOv8n via the `ultralytics` package. Auto-detects CUDA (`--device` overridable to force CPU). Sets the same `KMP_DUPLICATE_LIB_OK` guard as `embedder.py`, independently, since this is a separate process that never imports that module.

`train_linear_probe.py`

Generates domain-randomized training crops from `data/reference/` (every source × rotation × mirror × N jittered variants), embeds them in batches (chunked — `embedder.embed()` itself runs its whole input as one forward pass, fine for a handful of per-drawing candidate crops but not for several thousand training crops in one batch on 4GB VRAM), and fits a `LogisticRegression`. Prints its own synthetic-validation accuracy with an explicit caveat that this number is not evidence of real performance.

`convert_roboflow_dataset.py`

Converts the (evaluated, ultimately not-shipped — see docs/08 §1.4b) Roboflow “Circuit Recognition” dataset into this project's label format. `CLASS_MAP` only includes the 4 of 8 source classes visually verified to match this project's taxonomy; `--max-images` subsamples (randomly, not first-N, since a first-N slice of Roboflow's own export risks low diversity if near-duplicate augmented variants cluster together in export order) to keep a found dataset from swamping the class balance of the KiCad-based synthetic set it gets mixed into. Kept in the repository as a correct, reusable tool even though the specific training run it produced was reverted.

`validate_on_real.py`

Runs the full pipeline over every image in a directory and writes detection-overlay PNGs for manual visual audit — for any real drawing dropped in without hand-verified ground truth yet. Superseded by `edp`

`eval` specifically for D4/D5, which now have golden truth; this script remains the right tool for anything that doesn't.

config/

default.yaml

Every threshold in the pipeline, grouped by stage, each with an inline comment explaining *why* that specific value — not just what it is. Several values carry their own measured justification directly in the comment (`yolo_conf_threshold: 0.12 # lowered from 0.25 - was a recall bottleneck, verified on D5`) rather than a plain magic number, so the file itself is part of the project's "justify every decision" discipline, not just the prose docs.

reference_designators.yaml

The declarative OCR-designator/part-number table `classify/text_prior.py` reads. Two sections: `designators` (letter-prefix → class, some deliberately absent — `T` and `U` — with comments explaining exactly why a prior there would be actively wrong) and `part_number_families` (regex patterns for specific real part numbers, checked before designators since they're more specific evidence). Adding a new convention is a YAML edit, never a code change — the same "extend via data" principle applied to text evidence instead of visual evidence.

tests/

Deliberately scoped to the *deterministic* logic surrounding the GPU- backed models, not the models themselves (those are validated by `edp eval` against real output instead, since a unit test can't meaningfully assert "DINOv2 embeds this correctly").

- `test_classification_evidence.py` — fusion math (agreement beats a single strong disagreement, no-evidence sources are excluded not zero-padded, margin computation) and the OCR text-prior (normalization, the `$` → `S` substitution, exact-part-number vs. designator confidence tiers, ambiguous text correctly abstaining).
- `test_specialists.py` — the geometry specialists against this project's own reference renders (Capacitor→Capacitor, multi-cell Battery→Battery, Potentiometer has no circle, BJT has one), plus `select_specialist`'s routing table, including the explicit assertion that NPN/PNP is *not* reachable through it.
- `test_probe.py` — the linear-probe evidence source's abstain path (no artifact present) and its real-model path (a tiny synthetic `LogisticRegression`, not the real trained artifact, to stay fast and GPU-free).
- `test_library_cache.py` — the embedding-cache signature (stable when nothing changes, changes on a touched file or a changed rotation config), and cache hit/miss behavior on signature or metadata mismatch.
- `test_nets.py` — connectivity behavior at the net level (pre-dates this session; covers the core crossing-decomposition logic).
- `test_json_schema.py` — output-contract tests: every required field present, every `connections` reference points at a real symbol id in the same output.